The Errors Reviewers Catch: Statement-Level Autoformalization Errors and What Frontier Models Miss

Alexander Korbonits
Independent Researcher
alexkorbonits@uchicago.edu

Abstract

A formal statement can typecheck and admit a proof while failing to express the intended mathematics. We study such errors in the review history of a Lean 4 library of formalized open problems. From squash-merged pull requests, we recover 74 natural semantic errors among 360 changed statements, labeled by one annotator using the reviewer-approved corrections and comments. We evaluate twelve frontier models (eleven with scoreable output) with and without those corrections. The highest agreement with the reference labels is $F_1 = 0.79$ when corrections and comments are supplied, versus 0.56 for detection from the informal statement and candidate formalization alone. Wrong carriers and definitional mismatches are particularly difficult. Additional file context helps some models but does not close the gap. Among complete-coverage models, the same detection prompt yields $F_1 = 0.74$–1.00 on rule-written synthetic errors versus 0.29–0.52 on natural errors restricted to rule-supported classes. These results distinguish agreement given a correction from independent detection, and motivate review-derived evaluation alongside synthetic benchmarks. The single-source corpus and single-annotator labels limit generalization.

1 Introduction

Autoformalization systems translate informal mathematics into formal statements that can be checked and used by proof assistants [Wu et al., 2022, Azerbayev et al., 2023, Ying et al., 2024]. Typechecking or proving a statement does not establish that it expresses the intended mathematics. Audits have found statement defects in formal benchmarks [Ammanamanchi et al., 2026]. Faithfulness evaluation therefore requires a separate comparison with the informal source: existing approaches include reference-based equivalence checks [Poiroux et al., 2025], trained alignment scorers [Lu et al., 2025], and LLM judges [Zhang et al., 2026]. Human-annotated formalization attempts already exist; our contribution is to recover errors together with their corrections from *library review history*.

Our source is `formal-conjectures` [Firsching et al., 2026, The Formal Conjectures Authors, 2025], a Lean 4 [de Moura and Ullrich, 2021] library of formalized open problems. Its review process checks the relationship between mathematical docstrings and formal statements. We recover initial statements and reviewer-approved versions even when squash merging hides intermediate commits. The resulting corpus contains 5,726 documented statement pairs and 74 labeled natural errors.[1] We compare model judgments with reference labels under two conditions: *assisted explanation*, which supplies the correction and review comments, and *detection*, which supplies only the informal–formal

[1] Data, predictions, and code are included in the supplementary snapshot; repository: `https://github.com/korbonits/neurips-mathai-2026`.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: The 6th Workshop on Mathematical Reasoning and AI.

Docstring: *Can the smallest modulus of a covering system be arbitrarily large?*

```
theorem erdos_2 : answer(False) ↔                  -- first push
    ∃ B : ℕ, ∀ S, IsDistinctCoveringSystem S → minModulus S ≤ B

theorem erdos_2 : answer(False) ↔                  -- as merged
    ∀ B : ℕ, ∃ c : StrictCoveringSystem ℤ, ∀ i, ∃ m : ℕ,
      c.moduli i = Ideal.span {(m : ℤ)} ∧ B < m
```

Figure 1: A review-derived error (excerpt). The source docstring states that the answer is negative. The first push pairs `answer(False)` with the bounded form, reversing the intended polarity. The final reference assigns wrong-answer-marker and wrong-quantifier.

pair. Additional context and synthetic-error experiments test specific explanations for the performance gap.

Related task distinctions arise in software engineering. CodeReviewer separates change-quality estimation, review comment generation, and code refinement [Li et al., 2022]; Toggle separates bug localization from repair [Hossain et al., 2024]. These task distinctions motivate evaluating detection separately from performance when corrective information is supplied. Our experiments concern mathematical statement faithfulness; they do not establish that the observed gap is unique to Lean or equivalent to results on executable-software bugs.

2 A corpus of reviewed statement pairs

Extraction. We pair top-level declarations with their docstrings, strip proof bodies, and retain definition bodies, which specify the objects being defined. At the frozen repository commit, 5,726 of 6,949 declarations have docstrings. Records retain file, line, commit, and status information. A sampled extraction check compiles 20 of 20 statements; it does not certify the semantic faithfulness of the full corpus.

Mining corrections. Our shallow repository snapshot covers 502 squash-merged pull requests from 29 May to 31 August 2026. We recover the first pushed commit after each merge base and compare declaration statements with their merged versions, restricted to files touched by the squash commit. For 124 force-pushed pull requests, timeline queries recover pre-push commits for 18; 101 have garbage-collected commits, and five have no changes beyond the merge base. The procedure can therefore miss corrections already included in the earliest recoverable version. Proof-only and docstring-only changes are not candidates; the docstring at merge supplies the informal reference. The merged version and available review comments provide correction evidence, not an infallible semantic oracle.

Reference labels and revision. Mining yields 374 candidates, reduced to 360 by deduplication and rename detection. The author labeled these using a 14-class taxonomy, allowing multiple labels, with reviewer corrections and 528 inline comments available. The submitted version reported 105 semantic errors from a pass with model suggestions visible. For this version, a full blind pass over all 360 candidates supersedes those labels: model suggestions and prior labels were hidden, and both formalizations were elaborated in Lean where their difference was unclear. Forty-five preliminary errors became controls and 14 controls became errors, leaving **74 semantic errors** and 286 non-semantic changes. Mere omission of an `answer()` wrapper or reversal of equivalent sides of $\leftrightarrow$ is treated as non-semantic. The final set has 93 semantic label assignments across 12 occupied classes (Table 1). All reported natural-task scores use this reference. Appendix A documents the transition and its limitations.

3 Detection versus agreement given a correction

We evaluate twelve models, eleven with scoreable output, using frozen prompts, initial chunks of 12 records, and a 32k-token output budget; missing-response patches can use smaller batches. **Assisted**: the model sees the informal statement, first formalization, reviewer-approved correction, up to six review comments, machine triage bucket, and rename metadata. **Detection**: it sees only the informal statement and first formalization.

error class	meaning	records
wrong-type-or-domain	wrong carrier: $\mathbb{N}$ vs $\mathbb{Z}$ vs $\mathbb{Q}$ vs $\mathbb{R}$, meaning-changing coercion	20
definitional-mismatch	auxiliary definition does not capture the informal notion	15
missing-hypothesis	informal side condition absent (positivity, distinctness, $n \geq 1$)	12
wrong-answer-marker	`answer(True/False)` has incorrect polarity or wraps the wrong proposition	11
junk-value-trap	meaning shifts through Lean junk values: $x/0$, $\mathbb{N}$ subtraction, $\inf \emptyset$	11
wrong-quantifier	$\exists/\forall$ swapped, wrong scope or order	9
other	none of the above; explained in the record's note	6
wrong-connective	$\wedge/\vee$ / $\rightarrow$ / $\leftrightarrow$ misuse, misplaced negation	3
vacuous-or-trivial	trivially true or false (empty range, wrong index set)	2
extraneous-hypothesis	formal statement assumes more than the informal one	2
wrong-referent	points at the wrong declaration or constant	1
wrong-constant-or-bound	off-by-one, wrong constant, strict vs non-strict inequality	1

Table 1: Semantic classes in the final reference (74 records; multi-label). Bold labels identify the Lean-semantics group defined in Section 3.

	assisted					detection				
model	n	P	R	F1	κ	n	P	R	F1	κ
Claude Haiku 4.5	360	0.42	0.93	0.58	0.41	360	0.24	0.45	0.31	0.06
Claude Sonnet 5	360	0.56	0.97	0.71	0.61	360	0.70	0.38	0.49	0.41
Claude Opus 5	360	0.63	0.97	0.77	0.69	360	0.67	0.30	0.41	0.33
Claude Fable 5	360	0.68	0.95	0.79	0.73	360	0.61	0.51	0.56	0.46
GPT-5.6 Terra Pro	360	0.47	0.92	0.62	0.48	360	0.44	0.49	0.46	0.32
Gemini 3.1 Pro†	351	0.48	0.96	0.64	0.51	342	0.44	0.47	0.46	0.30
Kimi K3	360	0.66	0.85	0.75	0.67	360	0.54	0.45	0.49	0.37
Muse Spark 1.2	0	–	–	–	–	0	–	–	–	–
DeepSeek V4 Pro	360	0.42	0.93	0.58	0.41	360	0.35	0.58	0.44	0.25
Qwen 3.8 Max†	328	0.46	0.97	0.62	0.49	335	0.42	0.55	0.48	0.33
Grok 4.6	360	0.53	0.95	0.68	0.56	360	0.40	0.57	0.47	0.30
GLM 5.3†	328	0.40	0.92	0.56	0.39	319	0.43	0.43	0.43	0.29

Table 2: Agreement with the final reference on each condition's scoreable records. Columns n give answered counts; † marks models with incomplete coverage in either condition. Muse Spark 1.2 produced no scoreable output within the output budget. Missing outputs are not counted as correct or incorrect.

Both conditions request labels and explanations, but scoring measures binary semantic/non-semantic agreement (precision, recall, F_1, and Cohen's κ [Cohen, 1960]) and class agreement, not the correctness of explanations. Detection is a restricted statement-pair task, not a simulation of a reviewer with full file context and tools.

Overall results. Every model with scored outputs in both conditions has lower detection F_1 than assisted F_1 (Table 2; Figure 2). Claude Fable 5 leads both conditions, at 0.79 and 0.56, respectively. The highest detection recall is 0.58 (DeepSeek V4 Pro), whereas the highest precision is 0.70 (Claude Sonnet 5). The precision–recall tradeoff differs across models. Holding predictions fixed and using each model's shared answered records, assisted F_1 exceeds detection F_1 under both the submitted and final reference labels for all eleven scoreable models, with gaps of 0.18–0.44 under the submitted labels and 0.11–0.35 under the final labels (Appendix A).

Error classes. We group wrong-type-or-domain, definitional-mismatch, junk-value-trap, and wrong-answer-marker as the *Lean-semantics group*: 57 label assignments across potentially overlapping records. This grouping includes the project-specific answer-marker convention. Across models, the largest correct-class counts, selected separately for each class, are 5 of 20 wrong-type-or-domain errors, 4 of 15 definitional mismatches, 7 of 11 junk-value traps, and 6 of 11 answer-marker errors (Table 5). The first two classes are particularly difficult. Some smaller classes have much higher recall, but counts of one to three examples cannot support a robust class ranking.

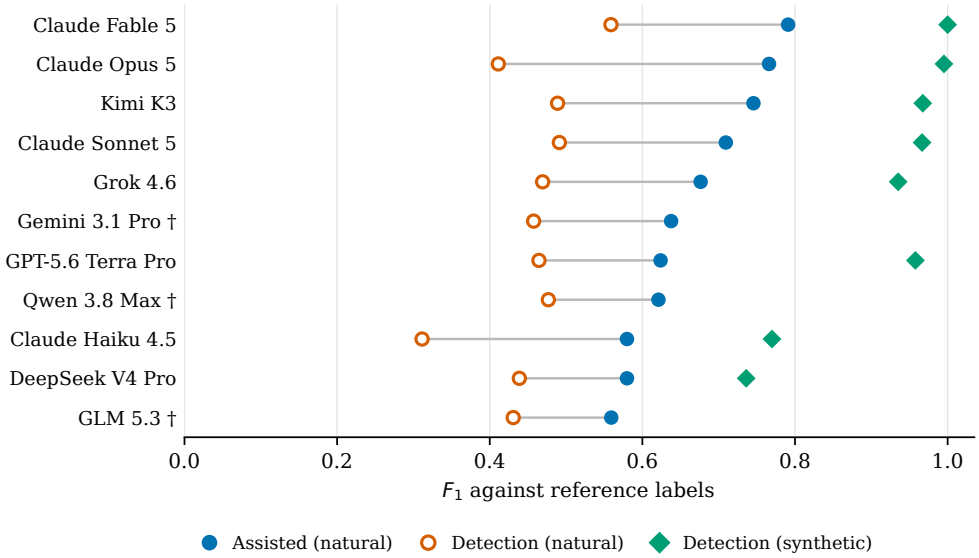


Figure 2: Per-model F_1 with corrective information (assisted), without it (detection), and on rule-written synthetic errors. Natural points use each condition's answered records from the full 360-candidate set; † marks incomplete natural coverage. Synthetic diamonds show only complete 359-record runs and use a different population and error prevalence. Muse has no scored natural output. Table 3 gives the rule-supported natural comparison and incomplete synthetic runs.

4 What changes with context and synthetic errors?

File context. Seven models were additionally evaluated with imports, namespace information, and file-local auxiliary declarations recovered from each candidate's *before* commit. Dependencies are followed for two hops, excluding the target declaration; the prompt also adds hazard-oriented guidance. This is a combined context-and-guidance intervention, not a clean ablation of context alone, and it does not supply all core or Mathlib definitions. Detection F_1 changes from -0.01 to $+0.14$ across models; no augmented score exceeds the best unaugmented score. Opus benefits most, while the overall detection gap remains (Table 4; protocol details in Appendix C).

Synthetic comparison. We also test the same detection prompt on 104 rule-written, typecheck-filtered perturbations of reviewed statements, with 255 unperturbed reviewed statements as controls. The source statements are outside the natural candidate set. Rules include quantifier and carrier changes, omitted hypotheses, and junk-value errors; they do not cover wrong referents, definitional mismatches, or the catch-all class. The class mix was constructed from preliminary labels, so we compare against a natural subset restricted to classes supported by the rules, not a frequency-matched or difficulty-matched population. Among models answering the entire synthetic set, synthetic F_1 exceeds this natural baseline by 0.35–0.60 (Table 3). Different controls and error prevalence limit the interpretation of this comparison. Nevertheless, success on these injected errors does not predict similarly high performance on the review-derived errors tested here.

Intermediate assistance. Removing review comments while retaining the correction reduces assisted F_1 by 0.04–0.08 for seven models. We also test oracle hints that an error exists or that name its class; both disclose the binary label and therefore cannot be interpreted as independent detection. Class hints yield higher class agreement, but a wrong-hint control elicits substantial compliance with incorrect labels (Appendix E).

Model	Natural, supported classes		Rule-written		
	n	F_1	n	F_1	ΔF_1
Claude Haiku 4.5	339	0.29	359	0.77	+0.48
Claude Sonnet 5	339	0.45	359	0.97	+0.51
Claude Opus 5	339	0.40	359	1.00	+0.60
Claude Fable 5	339	0.52	359	1.00	+0.48
GPT-5.6 Terra Pro	339	0.46	359	0.96	+0.50
Kimi K3	339	0.43	359	0.97	+0.54
DeepSeek V4 Pro	339	0.39	359	0.74	+0.35
Grok 4.6	339	0.40	359	0.94	+0.54
Incomplete coverage: descriptive scores only					
Gemini 3.1 Pro†	321	0.44	352	0.92	+0.49
GLM 5.3†	299	0.39	37	0.83	+0.44

Table 3: Detection on natural errors of rule-supported classes and rule-written synthetic errors. Complete populations contain 339 and 359 records. † marks incomplete coverage; GLM answered only 37 synthetic records. Main-text ranges use complete rows. Qwen and Muse are omitted because neither produced a scoreable synthetic response.

model	n	detection	context-augmented				Lean-semantics recall	
		F1	P	R	F1	ΔF1	detection	context-augmented
Claude Haiku 4.5	360	0.31	0.34	0.47	0.40	$+0.09$	4/57	6/57
Claude Sonnet 5	360	0.49	0.71	0.43	0.54	$+0.05$	8/57	11/57
Claude Opus 5	360	0.41	0.65	0.47	0.55	$+0.14$	8/57	16/57
Claude Fable 5	360	0.56	0.55	0.55	0.55	0.00	17/57	19/57
GPT-5.6 Terra Pro	360	0.46	0.49	0.53	0.51	$+0.04$	15/57	18/57
Kimi K3	360	0.49	0.45	0.50	0.47	-0.01	14/57	17/57
Grok 4.6	360	0.47	0.43	0.55	0.48	$+0.01$	17/57	20/57

Table 4: Detection with file context versus unaugmented detection. All seven context runs score 360 records. Pooled Lean-semantics recall counts correct assignments in the Lean-semantics group defined in Section 3.

5 Discussion and limitations

Models agree more closely with the reference labels when corrective evidence is supplied than when they must detect errors from statement pairs alone. High agreement when a fix is visible cannot establish reliable detection, nor does it certify an explanation's reasoning. Review-derived examples complement synthetic tests by exposing failures on errors encountered in mathematical practice. Potential uses include testing statement reviewers and prioritizing human inspection; benchmark precision should not be transferred directly to deployment, where the prevalence and kinds of errors may differ.

All records come from one repository and one review culture. Only merged pull requests with recoverable corrections enter the evaluation. A separate mining pass finds 111 declarations changed by 69 fix-marked commits within the same time window; these are descriptive post-merge changes, not a second labeled evaluation or an estimate of review's miss rate. Public availability and commit dates do not establish absence of training-data contamination.

All reference labels come from one annotator. Claude Fable 5, which leads both evaluation conditions, also generated the preliminary labeling suggestions. The full blind pass removes visible model suggestions but does not establish independent annotation reliability; greater Lean experience and use of elaboration also changed the procedure. Shared reference labels do not make systematic errors harmless or equally influential across models. Records cluster within pull requests, classes have small samples, and results are single runs. Output omissions, provider routing, and chunk composition may affect the scores. The evidence supports conclusions about these prompts, records, and reference labels, not about all mathematical review. Independent expert annotation and interactive Lean-assisted review remain important next steps.

References

Pawan Sasanka Ammanamanchi, Siddharth Bhat, and Stella Biderman. Faults in our formal benchmarking: Dataset defects and evaluation failures in Lean theorem proving, 2026. arXiv:2606.29493.

Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics, 2023. arXiv:2302.12433.

Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *CADE*, pages 625–635, 2021.

Moritz Firsching, Paul Lezeau, Salvatore Mercuri, Miklós Z. Horváth, Yaël Dillies, Calle Sönne, Eric Wieser, Fred Zhang, Thomas Hubert, Blaise Agüera y Arcas, and Pushmeet Kohli. Formal conjectures: An open and evolving benchmark for verified discovery in mathematics, 2026. arXiv:2605.13171.

Soneya Binta Hossain, Nan Jiang, Qiang Zhou, Xiaopeng Li, Wen-Hao Chiang, Yingjun Lyu, Hoan Nguyen, and Omer Tripp. A deep dive into large language models for automated bug localization and repair, 2024. arXiv:2404.11595.

Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu, and Neel Sundaresan. Automating code review activities by large-scale pre-training. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022.

Jianqiao Lu, Yingjia Wan, Yinya Huang, Jing Xiong, Zhengying Liu, and Zhijiang Guo. FormalAlign: Automated alignment evaluation for autoformalization. In *ICLR*, 2025.

Auguste Poiroux, Gail Weiss, Viktor Kunčak, and Antoine Bosselut. Reliable evaluation and benchmarks for statement autoformalization. In *EMNLP*, 2025.

The Formal Conjectures Authors. The Formal Conjectures Repository. `https://github.com/google-deepmind/formal-conjectures`, 2025.

Yuhuai Wu, Albert Qiaochu Jiang, Wenda Li, Markus Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *NeurIPS*, 2022.

Huaiyuan Ying, Zijian Wu, Yihan Geng, Zheng Yuan, Dahua Lin, and Kai Chen. Lean workbook: A large-scale Lean problem set formalized from natural language math problems, 2024. arXiv:2406.03847.

Ke Zhang, Patricio Gallardo Candela, Sudhir Murthy, Yi Xie, Zhi Wang, and Maziar Raissi. Beyond compilation: Evaluating faithful natural-language-to-Lean statement formalization, 2026. arXiv:2606.31002.

A Reference-label revision and reproducibility

The supplementary package includes the workshop data, predictions, prompts, and code, alongside a current results summary. The same release is available at `https://github.com/korbonits/neurips-mathai-2026`. The final reference is stored under `artifact/data/`; the source records file checksums in `artifact-version.json`, which the build verifies before regenerating tables. The submitted 105-error labels are included separately for the label-version sensitivity check. Model runs were conducted in August–September 2026. Predictions that do not depend on reference labels were rescored from disk after the full blind pass; label-dependent hint conditions were repatched on changed class sets.

For the label-version sensitivity check, we hold the current assisted and detection predictions fixed and restrict each model to the intersection of records answered in both conditions. We rescore these same records against the submitted reference at artifact commit `478ffcc` and the final reference. All eleven models retain higher assisted than detection F_1 under both references: the gaps range from 0.18 to 0.44 under the submitted labels and from 0.11 to 0.35 under the final labels. The supplementary robustness report gives per-model counts and scores; this checks sensitivity to the label revision, not annotation correctness.

The preliminary subset check reported 23/40 binary self-agreement (57%). The subsequent full blind pass agrees with the preliminary pass on 301/360 binary judgments ($\kappa = 0.57$), and on 273/360 exact class sets. The 57% subset agreement and the pass-to-pass κ of 0.57 are different statistics. The full pass used more Lean experience and elaboration of unclear cases, so pass-to-pass agreement is not identical-protocol test–retest reliability. Neither check supplies an independent second annotator. The revised labeling also treats purely conventional answer-wrapper changes as non-semantic. The final reference supersedes the preliminary labels rather than averaging the two passes. Of the 14 taxonomy options, non-semantic identifies controls, informal-changed is unoccupied, and 12 semantic classes are represented.

Natural candidates retain the reviewer-approved version and comment evidence; these support auditing the labels but do not prove faithfulness to the informal source. The docstring is taken at merge, so matching that version may differ from reconstructing the author's original intent. Eight candidates have no docstring (all are controls in the final reference); the frozen prompt instructs the model to judge the remaining fields and note this absence. They remain in the reported evaluation. Error directions are available in the artifact but are not evaluated here. Supplementary files include frozen prompts, final examples, and a description of score provenance.

LLM use disclosure

LLM assistants were used for tooling, analysis code, and manuscript drafting and revision. Claude Fable 5 generated the initial labeling suggestions and is also an evaluated model. The final pass hid suggestions and prior labels, but previous exposure and use of a single annotator remain limitations. The camera-ready revision used an assistant to audit saved results, regenerate tables, and revise the text. Model outputs are evaluated against human reference labels; no claim of independent expert annotation is made.

B Class-level detection

error class (n)	Claude Haiku 4.5	Claude Sonnet 5	Claude Opus 5	Claude Fable 5	GPT-5.6 Terra Pro	Gemini 3.1 Pro†	Kimi K3	DeepSeek V4 Pro	Qwen 3.8 Max†	Grok 4.6	GLM 5.3†
wrong-type-or-domain (20)	0	2	2	2	4	4	2	1	4/15	5	2/16
definitional-mismatch (15)	0	2	1	4	3	2	3	1	3/14	3	2/14
missing-hypothesis (12)	5	6	5	8	7	5	4	5	8/11	5	3/9
junk-value-trap (11)	2	3	3	6	5	3	5	2	7	3	4/10
wrong-answer-marker (11)	2	1	2	5	3	5	4	6	5	6	3/8
wrong-quantifier (9)	2	3	2	6	4	3	2	2	3/7	5	3/8
other (6)	0	0	0	0	0	0	0	0	0	0	0
wrong-connective (3)	3	1	1	1	2	1	1	2	2	1	1/2
extraneous-hypothesis (2)	1	1	1	0	1	1	0	0	1	2	2
vacuous-or-trivial (2)	0	2	2	2	1	1	2	0	2	2	1
wrong-constant-or-bound (1)	0	1	1	1	1	1	1	0	1	1	–
wrong-referent (1)	1	1	1	1	1	1	1	1	1	1	1

Table 5: Correct-class detection counts. Row totals count reference examples. For incomplete models (†), found/answered is shown when the answered count is below the row total; a dash denotes no answered examples. Bold labels identify the Lean-semantics group. Small classes are descriptive only.

The Lean-semantics group is defined in Section 3. Class recall requires assigning the correct class, not merely detecting that a statement contains some error. No evaluated detector assigns the catch-all `other` label to any of its six reference examples; this need not mean all six evade binary detection.

C Context and correction only conditions

The context builder reads each file at its before commit and follows local identifiers through two dependency hops. The declaration budget is 6,000 characters. Imports and namespace headers do not supply the definitions of all imported names. The added instruction also points models to possible semantic hazards, so effects cannot be attributed solely to extra code.

		assisted	correction only				Lean-semantics recall	
model	n	F1	P	R	F1	ΔF1	assisted	correction only
Claude Haiku 4.5†	359	0.58	0.38	0.97	0.54	−0.04	27/57	22/57
Claude Sonnet 5	360	0.71	0.48	0.97	0.64	−0.07	38/57	37/57
Claude Opus 5	360	0.77	0.56	0.97	0.71	−0.05	40/57	41/57
Claude Fable 5	360	0.79	0.59	0.95	0.73	−0.07	41/57	35/57
GPT-5.6 Terra Pro	360	0.62	0.38	0.96	0.54	−0.08	40/57	43/57
Kimi K3	360	0.75	0.54	0.96	0.69	−0.06	32/57	35/57
Grok 4.6	360	0.68	0.45	0.95	0.61	−0.07	44/57	38/57

Table 6: Keeping the corrected statement but removing review comments. The correction only condition retains other assisted metadata; n gives answered counts and † marks incomplete coverage. This intervention is not a reviewer-comments-only condition.

D Synthetic errors

Each injected statement was typechecked in its original file. Typechecking does not independently establish the intended semantic error or its class. Rule transformations and their source records are recorded in `scripts/make_synthetic_negatives.py` and `data/negatives_synthetic.jsonl` in the pinned artifact. The natural comparison keeps controls and only semantic records whose labels all occur in the rule-written set (339 records: 53 errors and 286 controls). This matches class support, not frequencies, control construction, or error prevalence. Partial-output runs are reported separately from complete ones; missing predictions are not scored as successes or failures.

This release includes the rule-written synthetic experiment reported here. The paper does not claim that every possible synthetic benchmark is easier than natural errors or that specialized alignment systems were evaluated.

E Oracle hints and incorrect-hint compliance

		blind		error-present		class named	
model	n	class set	Lean-sem.	class set	Lean-sem.	class set	Lean-sem.
Claude Haiku 4.5†	359	0.52	4/57	0.82	17/57	0.92	48/57
Claude Sonnet 5†	293	0.79	5/45	0.84	14/45	0.98	44/45
Claude Opus 5†	285	0.83	4/35	0.87	17/35	0.94	34/35
Claude Fable 5†	253	0.78	12/43	0.85	17/43	0.92	41/43
GPT-5.6 Terra Pro	360	0.71	15/57	0.82	26/57	0.92	51/57
Grok 4.6	360	0.66	17/57	0.83	26/57	0.92	50/57

Table 7: Blind detection, an error-existence hint, and a class hint, on each model's shared answered records (n); † marks incomplete coverage. Both hints disclose the binary label. Class-set agreement includes controls; pooled class recall uses reference labels in the Lean-semantics group (Section 3). Different row populations preclude interpreting this as a model ranking.

	controls	errors, wrong class hinted		errors, gold class hinted
model	hint rejected	wrong class echoed	a gold class found	a gold class found
Claude Haiku 4.5	70/283	55/74	7/74	66/74
Claude Sonnet 5	33/243	50/74	11/74	72/74
Claude Opus 5	100/177	41/70	25/70	60/64
Claude Fable 5	73/90	40/73	26/73	67/74
GPT-5.6 Terra Pro	184/286	27/74	26/74	61/74
Kimi K3	30/67	45/73	18/73	63/72
Grok 4.6	165/286	39/74	26/74	62/74

Table 8: Wrong-hint control. Each record receives a class absent from its reference label set. Denominators reflect available responses. Correct-hint agreement can coexist with acceptance of incorrect labels.

Hint conditions were repatched where the final class set changed. The existence-hint condition remains incomplete for several models, so the first table compares only shared answered records within each model. Kimi is omitted from that comparison because its existence-hint responses have insufficient coverage. Wrong-hint responses also have uneven coverage; they measure observed compliance, not a population-wide failure rate. Explanations were not independently judged for correctness.